Task Fifteen - Darn SQL Injection

Introduction

The goal of this report is to demonstrate SQL Injection attacks at Low, Medium, and High security levels in DVWA and document the results. Each level highlights different protections and shows how attackers adapt their techniques, from simple UNION queries to blind SQL injection methods.

Environment Setup

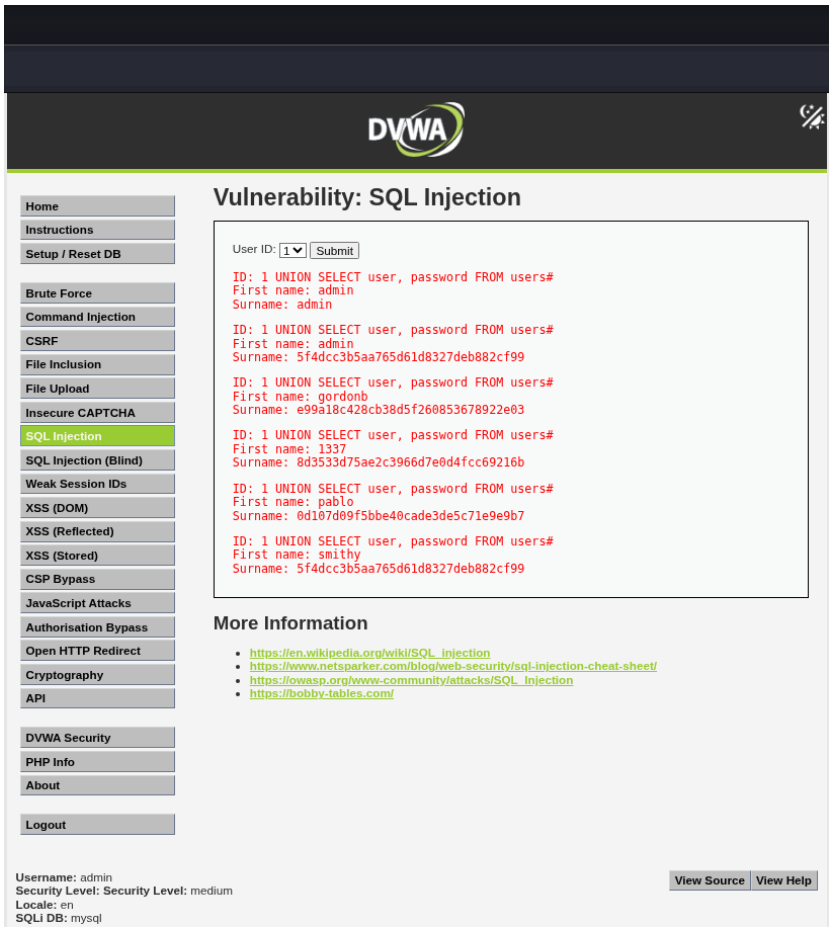
DVWA was deployed locally on Kali Linux with a PHP/MySQL backend. Security levels were tested at Low, Medium, and High. The experiments were conducted using the Firefox browser for interaction and Burp Suite for intercepting and modifying HTTP requests.

Low Security

- **Description**: Input is not sanitized, direct injection possible.
- **Payloads**: 1' OR '1'='1' #
- **Screenshot** :

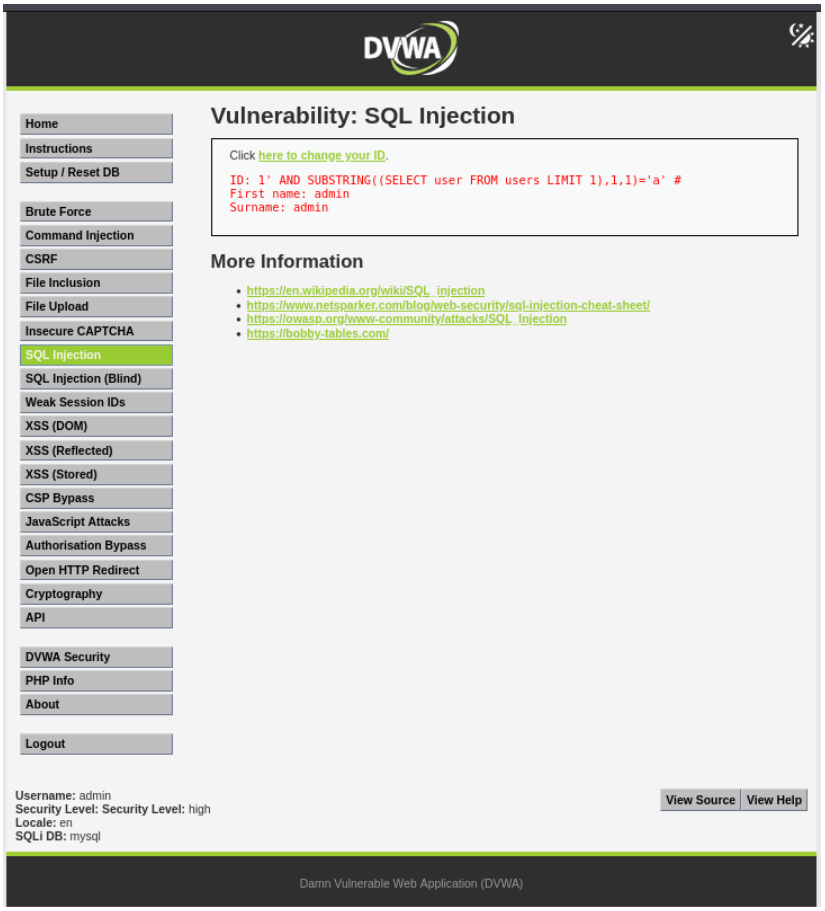
Medium Security

- **Description:** Input restricted to drop-down, POST request used.
- **Payloads:** id=1' UNION SELECT user, password FROM users #&Submit=Submit
- **Screenshot:**



High Security

- **Description:** Prepared statements block direct UNION injection. Blind SQLi required.
- **Payloads:** 1' AND SUBSTRING((SELECT user FROM users LIMIT 1),1,1)='a' #
- **Screenshots:**



Conclusion

At Low security, direct UNION injection easily exposed all users. At Medium**,** editing the POST body achieved the same result. At High, prepared statements blocked direct listing, and only blind SQL injection could infer data step by step. This shows that while simple protections stop basic attacks, secure coding practices like prepared statements are essential to defend against SQL injection.